

Lecture: Data Quality Check and Cleaning

Actuarial Data Science - Open Learning Resource
Fei Huang, UNSW Sydney



Table of contents

- Tibbles
- Import Data
- Tidy Data
- Assessing Data Quality
- Exploratory Data Analysis (EDA)



Recommended Reading

- *R for Data Science*, Chapters 5, 7, 10
- Applied predictive Modelling, Chapters 3.3 (only transformations to resolve outliers), 3.4

Some coding examples in this lecture are adapted from *R for Data Science* (Wickham, Çetinkaya-Rundel, and Grolemund 2023).



Tibbles



Tibbles

- We work with “tibbles” instead of R’s traditional `data.frame` in the `tidyverse` environment.
- The `tibble` package provides opinionated data frames that make working in the `tidyverse` easier.
- See `vignette("tibble")` for more information.

This lecture focuses on data quality: how to store your data in a tidy, convenient format, and how to identify and fix common data issues before they silently undermine your analysis. The aim is that by the end, you feel more confident trusting the datasets you work with, or at least knowing when you *should not* trust them yet.

```
1 #install.packages("tidyverse")  
2 library(tidyverse)
```



Creating Tibbles

- Convert a data frame to a tibble: `as_tibble()`
- Create tibbles: `tibble()`
- Use non-syntactic names with backticks
- `tribble()`: create tibbles in a transposed format



Example: Converting to a Tibble

```
1 as_tibble(iris)
```



Sepal.Length <dbl>	Sepal.Width <dbl>	Petal.Length <dbl>	Petal.Width <dbl>
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.2
5.4	3.9	1.7	0.4
4.6	3.4	1.4	0.3
5.0	3.4	1.5	0.2
4.4	2.9	1.4	0.2
4.9	3.1	1.5	0.1

1-10 of 150 rows | 1-4 of 5 columns

Previous 1 2 3 4 5 6 ... 15 Next



Example: Creating a Tibble

```
1 tibble(  
2   x = 1:5,  
3   y = 1,  
4   z = x ^ 2 + y  
5 )
```

x	y	z
<int>	<dbl>	<dbl>
1	1	2
2	1	5
3	1	10
4	1	17
5	1	26

5 rows



Example: Creating Tibbles with Special Names

```
1 tibble(  
2   `:)` = "smile",  
3   ` ` = "space",  
4   `2000` = "number"  
5 )
```

:) <chr>	<chr>	2000 <chr>
smile	space	number

1 row

```
1 tribble(  
2   ~x, ~y, ~z,  
3   #--|--|----  
4   "a", 2, 3.6,  
5   "b", 1, 8.5  
6 )
```

x <chr>	y <dbl>	z <dbl>
a	2	3.6
b	1	8.5

2 rows



Pipe Data using %>%

- Use %>% to emphasise a sequence of actions rather than the object being acted on
- Pronounce %>% as “then” when reading code
- No need to create intermediate objects
- %>% should always have a space before it and is usually followed by a new line.

```
1 iris %>%
2   group_by(Species) %>%
3   summarise(
4     Sepal.Length = mean(Sepal.Length),
5     Sepal.Width = mean(Sepal.Width),
6     Species = n_distinct(Species)
7   )
```

Species <int>	Sepal.Length <dbl>	Sepal.Width <dbl>
1	5.006	3.428
1	5.936	2.770
1	6.588	2.974

3 rows



Tibble vs. Data Frame

- `tibble()` does less:
 - it never changes the type of the inputs (e.g. it does not convert strings to factors)
 - it never changes variable names
 - it never creates row names
- Printing
 - tibbles show only the first 10 rows
 - each column reports its type
 - use `print()` to display more rows (`n`) and columns (`width`)
- Subsetting
 - extract a single variable using `$` (by name) or `[[ ]]` (by name or position)
 - within a pipe `%>%`, use the special placeholder `.`



Examples: Extracting Variables from a Tibble

```
1 df <- tibble(  
2   x = runif(5),  
3   y = rnorm(5)  
4 )  
5 # Extract by name  
6 df$x
```

```
[1] 0.06613743 0.64441979 0.87036981 0.99049084 0.76861869
```

```
1 df[["x"]]
```

```
[1] 0.06613743 0.64441979 0.87036981 0.99049084 0.76861869
```

```
1 # Extract by position  
2 df[[1]]
```

```
[1] 0.06613743 0.64441979 0.87036981 0.99049084 0.76861869
```

```
1 df %>% .$x
```

```
[1] 0.06613743 0.64441979 0.87036981 0.99049084 0.76861869
```

```
1 df %>% .[[ "x" ]]
```

```
[1] 0.06613743 0.64441979 0.87036981 0.99049084 0.76861869
```



Import Data



Import Data

- Read [Introduction to Data Science](#), Chapter 5
- Read [R for Data Science](#), Chapter 7



Tidy Data



Tidy Data

- The same data can be organised in different ways.
- Tidy data is easier to work with.
- There are three interrelated rules that make a dataset tidy:
 - Each variable has its own column
 - Each observation has its own row
 - Each value has its own cell
- Practical guidelines:
 - Store each dataset as a tibble
 - Store each variable in its own column
- `dplyr`, `ggplot2`, and other `tidyverse` packages are designed to work with tidy data

```
1 library(tidyverse)
```



Pivoting: Longer

- A common problem occurs when column names represent values rather than variables.
- Example: `table4a`
 - Column names (1999, 2000) represent values of the `year` variable
 - Values in these columns represent values of the `cases` variable
 - Each row represents two observations, not one

```
1 #table4a
2
3 table4a %>%
4   pivot_longer(cols = c(`1999`, `2000`), names_to = "year", values_to = "cases")
```

country <chr>	year <chr>	cases <dbl>
Afghanistan	1999	745
Afghanistan	2000	2666
Brazil	1999	37737
Brazil	2000	80488
China	1999	212258
China	2000	213766

6 rows



Pivoting: Wider

- `pivot_wider()` is the opposite of `pivot_longer()`
- Use it when a single observation is spread across multiple rows.
- Example: `table2`
 - Each observation is a country in a given year
 - Data for each observation is spread across multiple rows

```
1 table2 %>%
2   pivot_wider(names_from = type, values_from = count)
```

country <chr>	year <dbl>	cases <dbl>	population <dbl>
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

6 rows



Separating

- `table3` has a different problem: one column (`rate`) contains two variables (`cases` and `population`).
- `separate()` splits one column into multiple columns, by splitting wherever a separator character appears.
- By default, `separate()` splits values at non-alphanumeric characters (i.e. characters that are not numbers or letters).
- Use the `sep` argument to specify a custom separator.



1 table3



country <chr>	year <dbl>	rate <chr>
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583

6 rows



Example: Using `separate()`

```
1 table3 %>%
2   separate(rate, into = c("cases", "population"), sep = "/")
```

country <chr>	year <dbl>	cases <chr>	population <chr>
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

6 rows

```
1 #table3 %>%
2 #   separate(rate, into = c("cases", "population"), sep = "/", convert=TRUE)
3
4 #table3 %>%
5 #   separate(year, into = c("century", "year"), sep = 2)
```



Unite

- `unite()` is the inverse of `separate()`: it combines multiple columns into a single column
- By default, it places an underscore (`_`) between values from different columns
- If no separator is desired, use `sep = ""`

```
1 table5 %>%
2   unite(new, century, year)
```

country <chr>	new <chr>	rate <chr>
Afghanistan	19_99	745/19987071
Afghanistan	20_00	2666/20595360
Brazil	19_99	37737/172006362
Brazil	20_00	80488/174504898
China	19_99	212258/1272915272
China	20_00	213766/1280428583

6 rows

```
1 #table5 %>%
2 #   unite(new, century, year, sep = "")
```



Missing Values

- A value can be missing in one of two ways:
 - **Explicitly:** represented by **NA** – *the presence of an absence*
 - **Implicitly:** not recorded in the dataset – *the absence of a presence*

```
1  stocks <- tibble(  
2    year   = c(2015, 2015, 2015, 2015, 2016, 2016, 2016),  
3    qtr    = c( 1,    2,    3,    4,    2,    3,    4),  
4    return = c(1.88, 0.59, 0.35,  NA, 0.92, 0.17, 2.66)  
5  )
```



Making Implicit Missing Values Explicit

- Use `pivot_wider()`
- Use `complete()`

```
1 stocks %>%
2   pivot_wider(names_from = year, values_from = return)
```

qtr <dbl>	2015 <dbl>	2016 <dbl>
1	1.88	NA
2	0.59	0.92
3	0.35	0.17
4	NA	2.66

4 rows

```
1 #stocks %>%
2 #   complete(year, qtr)
```



Making Explicit Missing Values Implicit

- Set `values_drop_na = TRUE` in `pivot_longer()` to convert explicit missing values into implicit ones

```
1 stocks %>%
2   pivot_wider(names_from = year, values_from = return) %>%
3   pivot_longer(
4     cols = c(`2015`, `2016`),
5     names_to = "year",
6     values_to = "return",
7     values_drop_na = TRUE
8   )
```

	qtr <dbl>	year <chr>	return <dbl>
	1	2015	1.88
	2	2015	0.59
	2	2016	0.92
	3	2015	0.35
	3	2016	0.17
	4	2016	2.66

6 rows



Filling Missing Values with `fill()`

- `fill()` replaces missing values with the most recent non-missing value (also known as last observation carried forward).

```
1 treatment <- tribble(
2   ~ person,      ~ treatment, ~response,
3   "Derrick Whitmore", 1,      7,
4   NA,              2,      10,
5   NA,              3,      9,
6   "Katherine Burke", 1,      4
7 )
8
9 treatment %>%
10  fill(person)
```

person <chr>	treatment <dbl>	response <dbl>
Derrick Whitmore	1	7
Derrick Whitmore	2	10
Derrick Whitmore	3	9
Katherine Burke	1	4

4 rows



Assessing Data Quality



Common Data Issues

- Missing data
- Irregular data and outliers
- Uninformative data
- Censored and truncated data
- High cardinality features
- Imbalanced data



Diagnosing Missing Data

- Descriptive statistics
- Visualisation (e.g. plots)
- See also: [Missing values in tidy data](#); and [Missing values in EDA](#)



Missing Data

- Understand why values are missing
- Distinguish between structurally missing values and other types
 - e.g. the number of children a man has given birth to
- **Informative missingness:** the pattern of missing data is related to the outcome
 - e.g. in a drug study, side effects are so severe that patients drop out



Handling Missing Values

- Include a missing indicator (dummy variable)
 - useful when the pattern of missingness is informative
- Some models can handle missing data (e.g. tree-based methods)
- Many models cannot handle missing values
 - linear models, neural networks, SVMs
- Remove observations or variables as a last resort
 - may be feasible for large dataset
- Impute missing values



Imputing Missing Values

- Use information from other predictors in the training data to estimate missing values
 - simple methods: mean, median or mode
 - model-based methods: e.g. k-nearest neighbours
- Imputation is extensively studied in the statistical literature for inference, but is often less critical in predictive modelling



Irregular Data and Outliers

- Detection:
 - Descriptive statistics
 - Visualisation (e.g. boxplots, scatter plots)
 - Outlier detection methods
- Handling:
 - Data validation: ensure there are no recording errors
 - Remove or adjust values
 - Outliers may belong to a different population
 - Use models that are robust to outliers (e.g. tree-based methods, SVMs)
 - Apply transformations to reduce the impact of outliers, such as the *spatial sign* transformation
 - each observation is scaled by its norm

Example: **Outliers**



Uninformative Data

- Repetitive data
- Duplicate observations
- Irrelevant variables
- Collinearity: two or more predictor variables are highly correlated
 - Redundant predictors increase model complexity
 - Can lead to unstable models, numerical issues, and degraded predictive performance
 - For linear regression, measures such as the variance inflation factor (VIF) can be used to identify predictors that are highly correlated with others



Censored data

- The value of an observation is only partially known
- For interpretation or inference
 - Typically handled using formal methods with assumptions about the censoring mechanism
- For prediction
 - Often treated as missing data or the censored value is used as observed



Examples:

General insurance (policy limits); life insurance (age groups of mortality data)

High-Cardinality Features

- Categorical predictors with many unique levels
- Examples: postcodes, medical condition codes, or similar variables



Some references on handling high-cardinality features:

- [Dealing with features that have high cardinality](#)
- [Encoding High-Cardinality String Categorical Variables](#)
- [Similarity Encoding for Learning with Dirty Categorical Variables](#)
- [Nonlife Insurance Risk Classification Using Categorical Embedding](#)
- [Using Random Effects to Account for High-Cardinality Categorical Features and Repeated Measures in Deep Neural Networks](#)



Imbalanced Data

- Imbalance between classes (e.g. control vs treatment) can cause modelling issues
- Construct a balanced training set to improve model performance
 - **Undersampling:** reduce the number of observations in the majority class
 - **Oversampling:** generate additional observations in the minority class



Data Validation

- Validate data against external sources and previous datasets
- Consult domain experts or data providers to assess data quality



Exploratory Data Analysis (EDA)



Exploratory Data Analysis

- Generate questions about your data
- Search for answers by visualising, transforming, and modelling your data
- Use what you learn to refine your questions and/or generate new ones
- We combine `dplyr` and `ggplot2` to interactively ask questions, answer them with data, and then ask new questions
- Two types of questions are especially useful for making discoveries in your data:
 - What type of variation occurs within variables?
 - What type of covariation occurs between variables?

```
1 library(tidyverse)
```



Adapted from Wickham, Çetinkaya-Rundel, and Grolemund (2023), see [Chapter 10](#) of R for Data Science



Variation

- Variation is the tendency of the values of a variable to change from measurement to measurement.
- Variables:
 - Continuous variable: can take any value within a range of possible values
 - Categorical variable: takes one of a fixed set of values



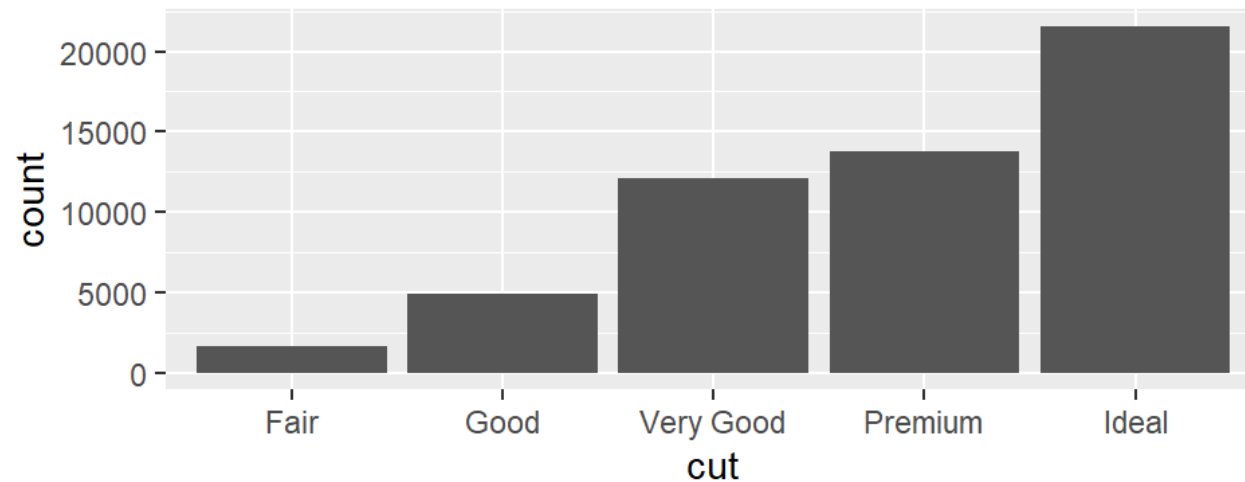
Visualising Distributions: Categorical Variables

- To examine the distribution of a categorical variable, use a bar chart
- Data: `diamonds` (see `?diamonds` for details)

See also: **Covariation**

```
1 ggplot(data = diamonds) +  
2   geom_bar(mapping = aes(x = cut))
```

```
1 #diamonds %>%  
2 #   count(cut)
```

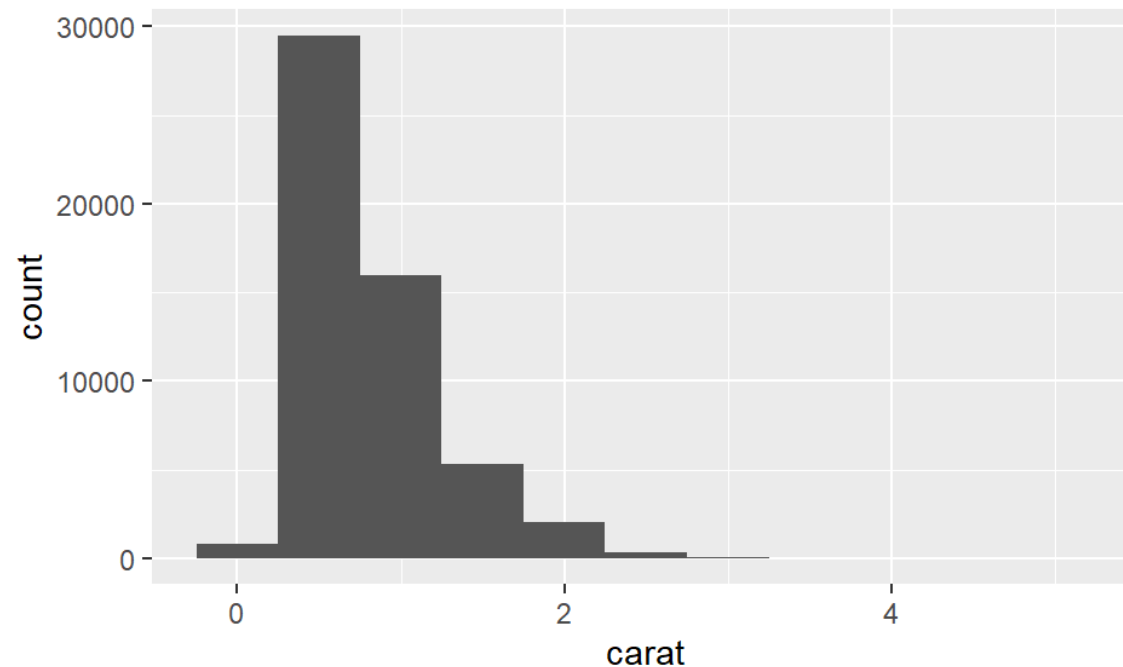


Visualising Distributions: Continuous Variables

- To examine the distribution of a continuous variable, use a histogram

```
1 ggplot(data = diamonds) +  
2   geom_histogram(mapping = aes(x = carat), binwidth = 0.5)
```

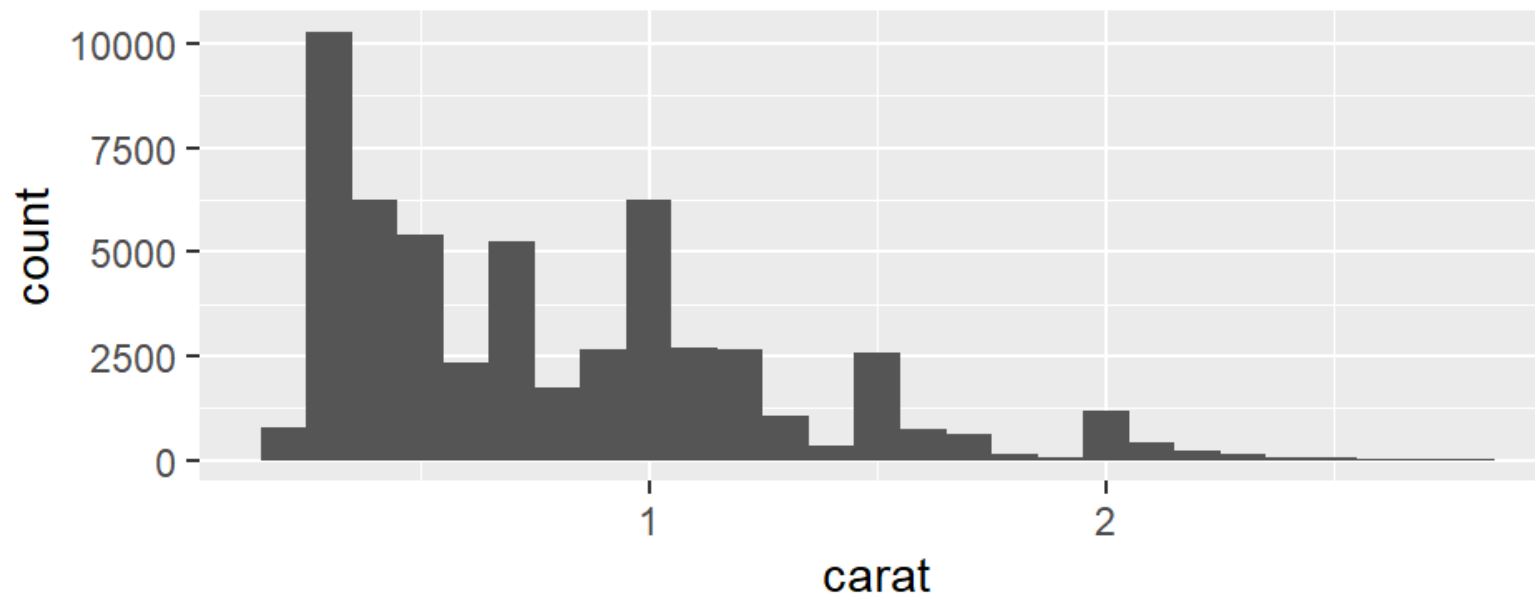
```
1 #diamonds %>%  
2 #   count(cut_width(carat, 0.5))
```



Exercise: Histogram Bin Widths

- Plot a histogram of diamonds with size less than 3 carats (using `filter()`), and use a smaller binwidth of 0.1.

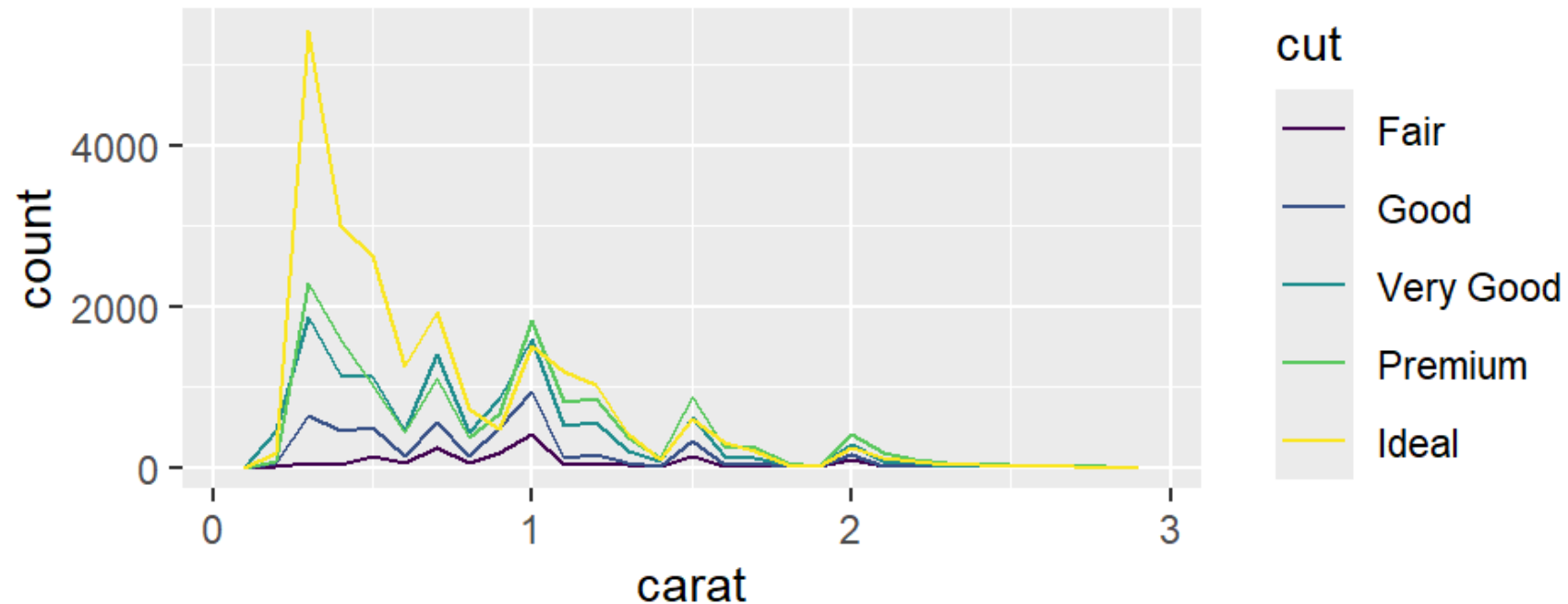
```
1 smaller <- diamonds %>%  
2   filter(carat < 3)  
3  
4 ggplot(data = smaller, mapping = aes(x = carat)) +  
5   geom_histogram(binwidth = 0.1)
```



Exercise: Frequency Polygons by Group

- Overlay multiple histograms in the same plot by `cut` using `geom_freqpoly()`

```
1 ggplot(data = smaller, mapping = aes(x = carat, colour = cut)) +  
2   geom_freqpoly(binwidth = 0.1)
```



See also: [Covariation](#)



Typical Values

- In both bar charts and histograms, tall bars show common values of a variable, while shorter bars show less common values. Gaps indicate values that do not appear in the data.
- To turn this information into useful questions, look for anything unexpected:
 - Which values are most common? Why?
 - Which values are rare? Why? Does that match your expectations?
 - Can you see any unusual patterns? What might explain them?

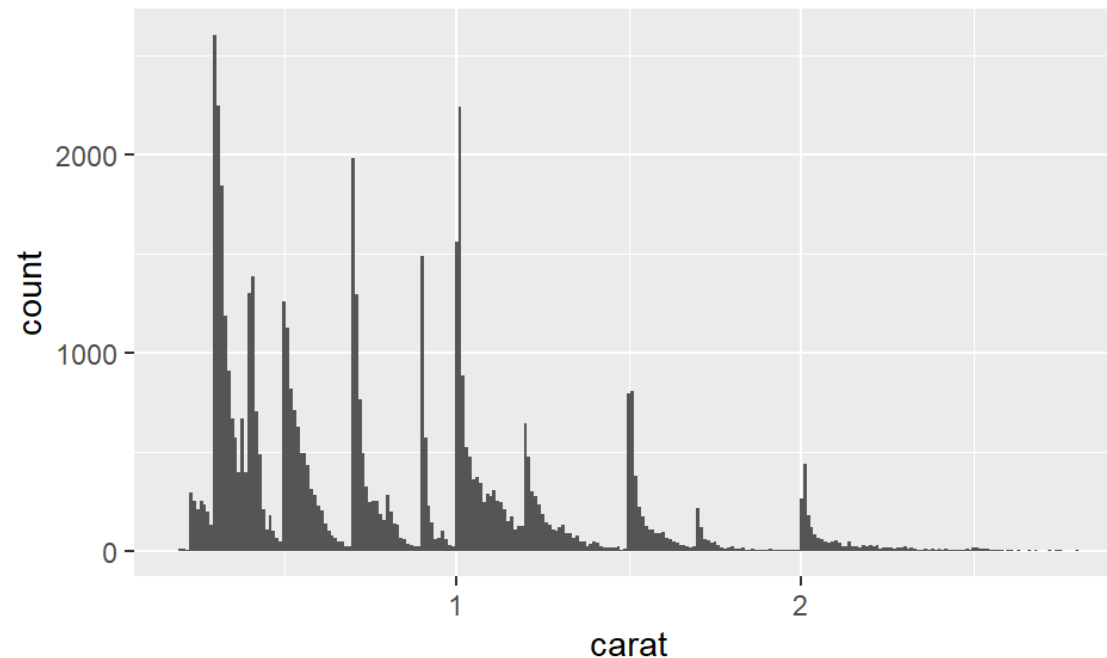


Example: Interpreting a Histogram

Question:

Look at the histogram below. What questions can you ask?

```
1 ggplot(data = smaller, mapping = aes(x = carat)) +  
2   geom_histogram(binwidth = 0.01)
```



Example: Discussion

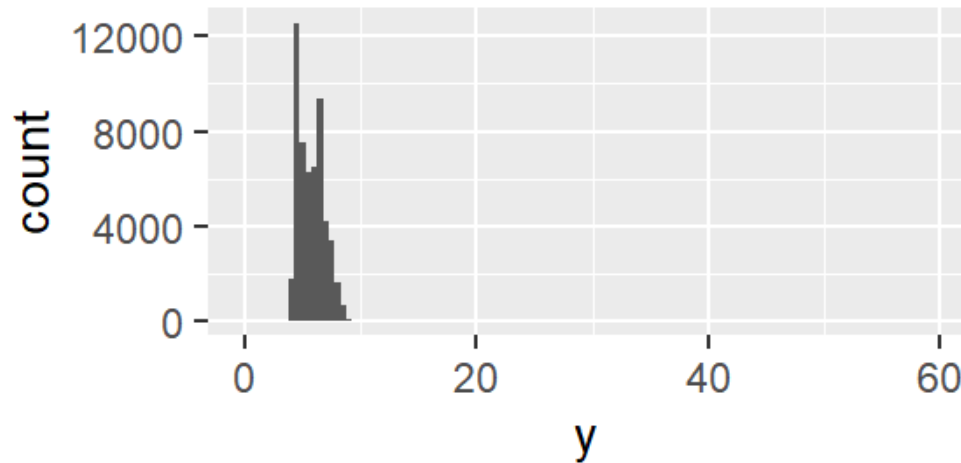
- Why are there more diamonds at whole carats and common fractional values?
- Why are there more diamonds just to the right of each peak than to the left?
- Why are there no diamonds larger than 3 carats?



Unusual Values (Outliers)

- Outliers are observations that are unusual; data points that do not seem to fit the overall pattern
- Sometimes outliers are data entry errors; other times they may reveal important insights
- When you have a large dataset, outliers can be difficult to see in a histogram

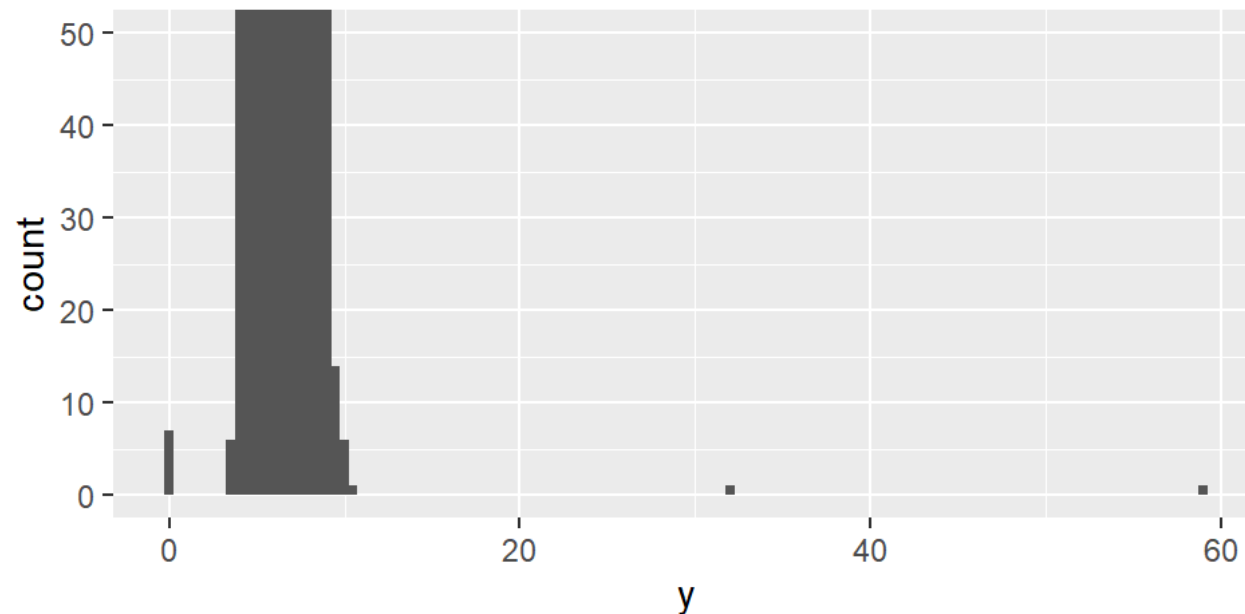
```
1 ggplot(diamonds) +  
2   geom_histogram(mapping = aes(x = y), binwidth = 0.5)
```



Visualising Outliers

- To make unusual values easier to see, we can zoom in using `coord_cartesian()`, often together with `ylim()` or `xlim()`, to restrict the axis ranges.

```
1 ggplot(diamonds) +  
2   geom_histogram(mapping = aes(x = y), binwidth = 0.5) +  
3   coord_cartesian(ylim = c(0, 50))
```



Display Unusual Values

- Extract them using `dplyr`.
- What questions might you ask about these observations?

```
1 unusual <- diamonds %>%
2   filter(y < 3 | y > 20) %>%
3   select(price, x, y, z) %>%
4   arrange(y)
5 unusual
```

price <int>	x <dbl>	y <dbl>	z <dbl>
5139	0.00	0.0	0.00
6381	0.00	0.0	0.00
12800	0.00	0.0	0.00
15686	0.00	0.0	0.00
18034	0.00	0.0	0.00
2130	0.00	0.0	0.00
2130	0.00	0.0	0.00
2075	5.15	31.8	5.12
12210	8.09	58.9	8.06

9 rows



Deal with Outliers: Example

- In the 'Diamond' example, the **y** variable measures one of the three dimensions of a diamond (in mm)
- Diamonds cannot have a width of 0 mm, so these values must be incorrect
- We might also suspect that measurements of 32mm and 59mm are implausible: such diamonds would be over an inch long, but do not cost hundreds of thousands of dollars



Deal with Outliers

- Repeat your analysis with and without the outliers.
- If they have minimal impact on the results and the cause is unclear, it is reasonable to replace them with missing values and proceed
- If they have a substantial impact on your results, do not remove them without justification
 - Investigate the cause (e.g. data entry errors)
 - Clearly document any decisions to remove them in your analysis



Missing Values

- If you encounter unusual values in your dataset and want to proceed with your analysis, you have two options:
 - Drop the entire row containing the unusual values (not recommended — why?)

```
1 diamonds2 <- diamonds %>%  
2   filter(between(y, 3, 20))  
3 #diamonds2
```

- Replace the unusual values with missing values (NA) using `mutate()` with `ifelse()` or `case_when()`

```
1 diamonds2 <- diamonds %>%  
2   mutate(y = ifelse(y < 3 | y > 20, NA, y))  
3 #diamonds2
```

- `ggplot2` does not include missing values in plots, but it will display a warning that they have been removed



Covariation

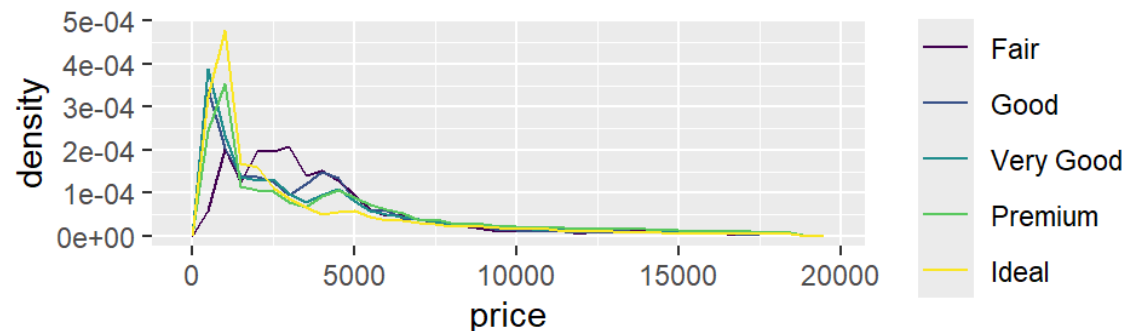
- If variation describes the behaviour within a variable, covariation describes behavior between variables.
- Covariation is the tendency for two or more variables to vary together in a related way
- The best way to identify covariation is to visualise relationships between variables.
- We consider three common cases:
 - A categorical and a continuous variable
 - Two categorical variables
 - Two continuous variables



A Categorical and a Continuous Variable

- Explore the distribution of a continuous variable across levels of a categorical variable
- Use `geom_freqpoly()` (e.g. see [frequency polygon example](#))
- It can be difficult to compare distributions when overall counts differ substantially (e.g. see [distribution of cut example](#)).
- Instead of displaying counts, we can display density, where the area under each curve is standardised to one

```
1 ggplot(data = diamonds, mapping = aes(x = price, y = ..density..)) +  
2   geom_freqpoly(mapping = aes(colour = cut), binwidth = 500)
```



Boxplot

- Another way to display the distribution of a continuous variable across categories is a **boxplot**.

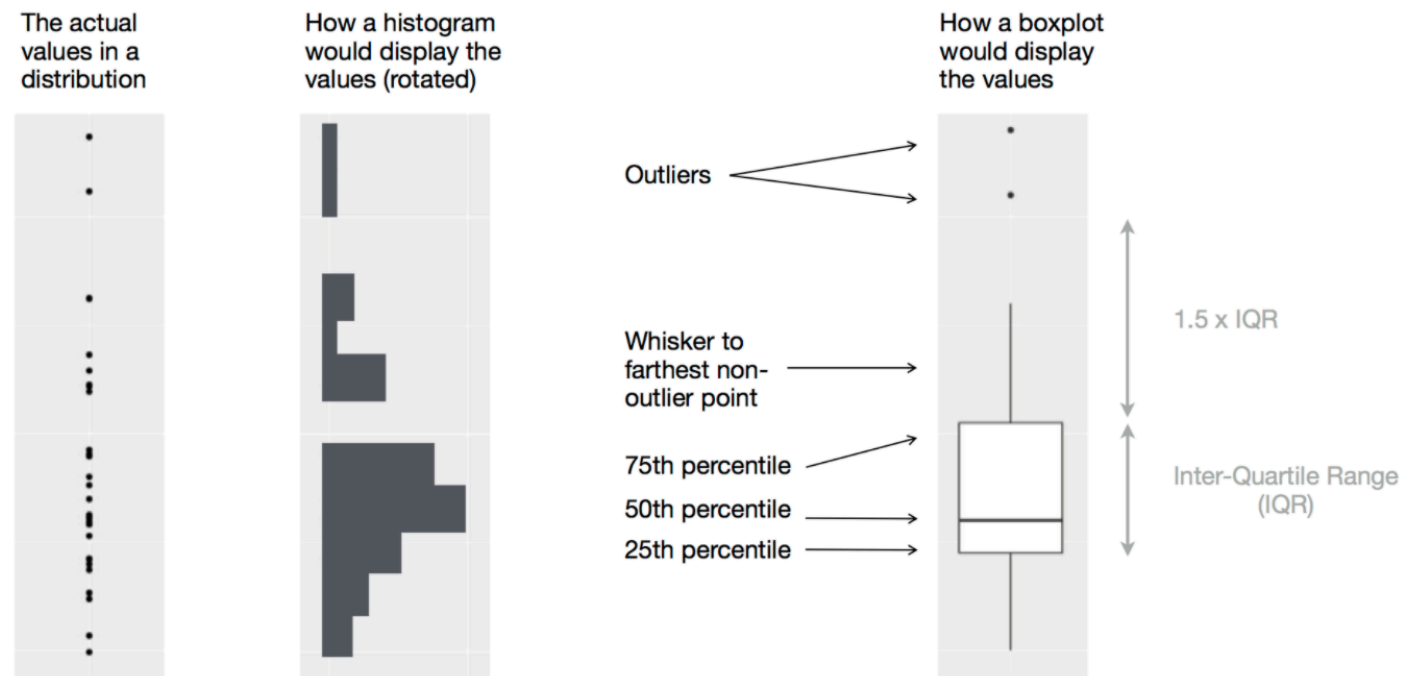
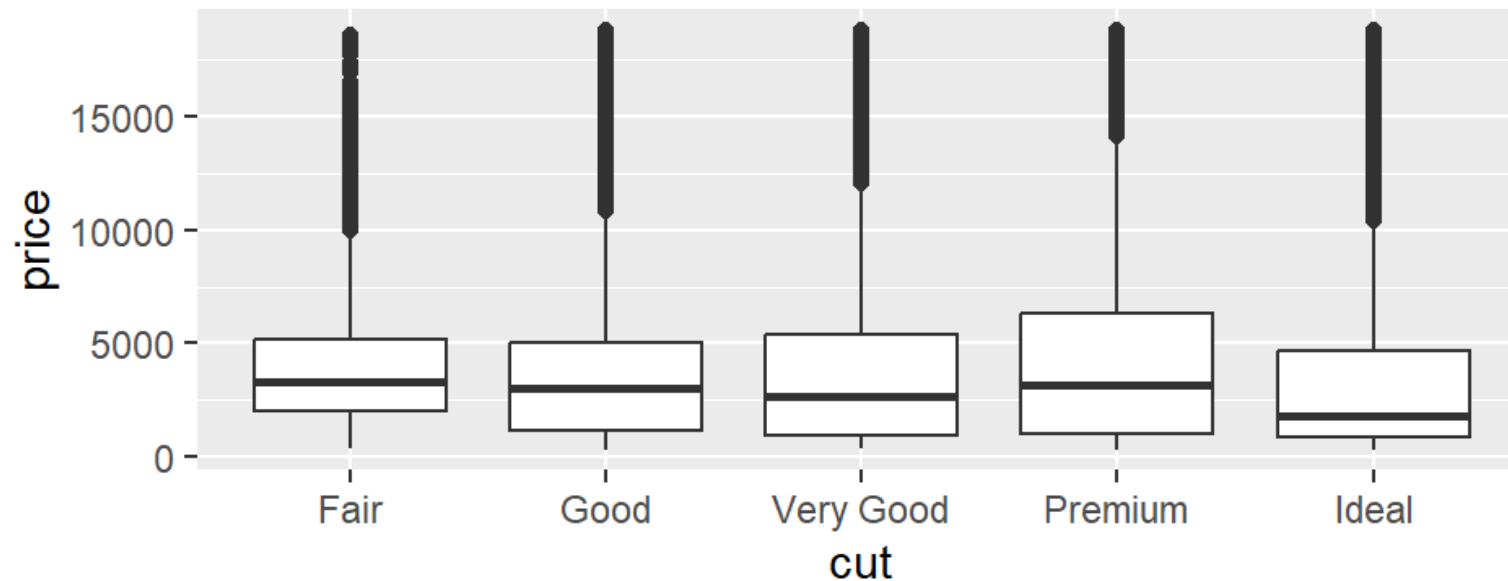


Diagram illustrating how a boxplot is constructed (Source: *R for Data Science* (Wickham, Çetinkaya-Rundel, and Grolemund 2023))

Example: Diamond Prices by Cut

- Examine the distribution of diamond prices by **cut**. What do you observe?

```
1 ggplot(data = diamonds, mapping = aes(x = cut, y = price)) +  
2   geom_boxplot()
```



- This suggests the counterintuitive finding that higher-quality diamonds are cheaper on average. Why might this be the case?

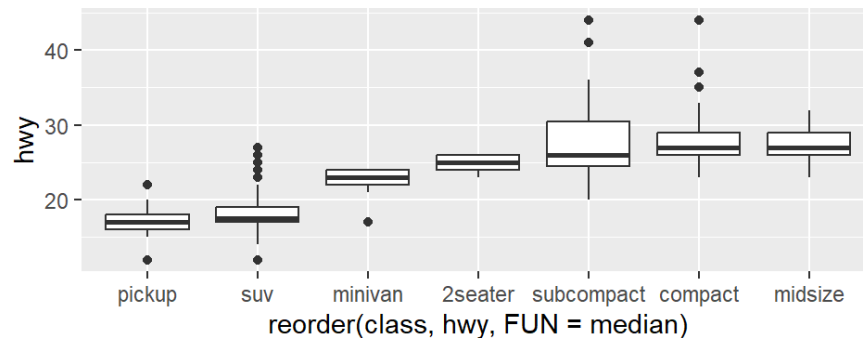


Example: Highway Mileage by Vehicle Class

- Consider the `mpg` dataset. We want to understand how highway mileage (`hwy`) varies across vehicle classes (`class`)
- To make patterns easier to see, reorder `class` by the median of `hwy` using `reorder(..., FUN = median)`
- For long variable names, `geom_boxplot()` may be clearer when flipped using `coord_flip()`

```
1 ggplot(data = mpg) +
2   geom_boxplot(mapping = aes(x = reorder(class, hwy, FUN = median), y = hwy)) #+
```

```
1 # coord_flip()
```



Two Categorical Variables

- To visualise covariation between categorical variables, count the number of observations for each combination.
- One approach is to use `geom_count()`

```
1 ggplot(data = diamonds) +  
2   geom_count(mapping = aes(x = cut, y = color))
```



Example: Computing Counts with dplyr

- Another approach is to compute counts using `dplyr`

```
1 diamonds %>%
2   count(color, cut)
```

color <ord>	cut <ord>	n <int>
D	Fair	163
D	Good	662
D	Very Good	1513
D	Premium	1603
D	Ideal	2834
E	Fair	224
E	Good	933
E	Very Good	2400
E	Premium	2337
E	Ideal	3903

1-10 of 35 rows

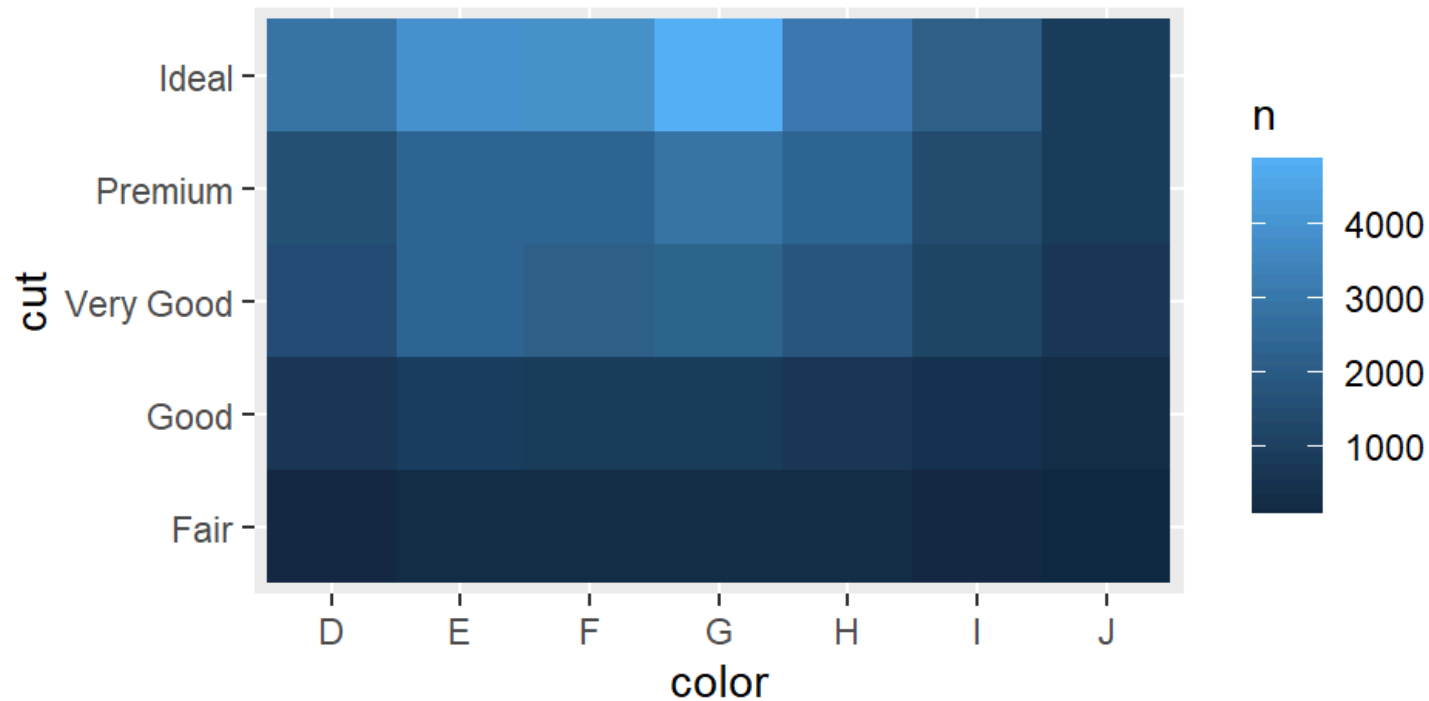
Previous 1 2 3 4 Next



Example: Visualising Counts with geom_tile()

- Then visualise the counts using `geom_tile()` with the fill aesthetic

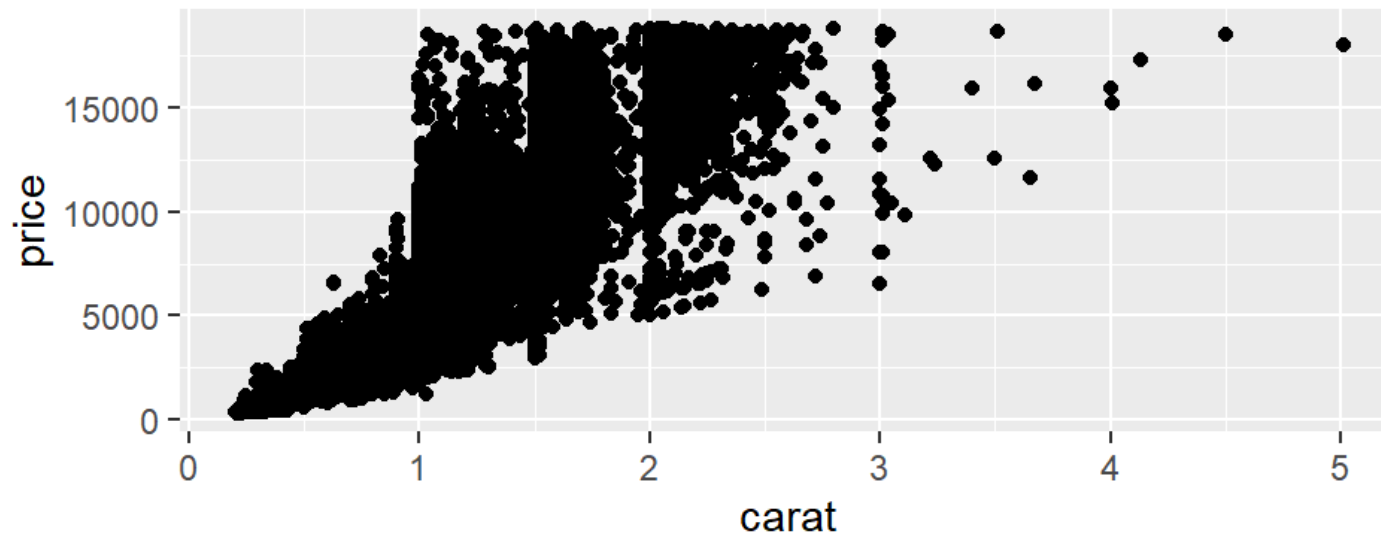
```
1 diamonds %>%  
2   count(color, cut) %>%  
3   ggplot(mapping = aes(x = color, y = cut)) +  
4     geom_tile(mapping = aes(fill = n))
```



Two Continuous Variables

- A common way to visualise the covariation between two continuous variables is a **scatter plot** using `geom_point()`
- Covariation appears as patterns in the points
- Example: visualise the relationship between carat size and price of diamonds

```
1 ggplot(data = diamonds) +  
2   geom_point(mapping = aes(x = carat, y = price))
```



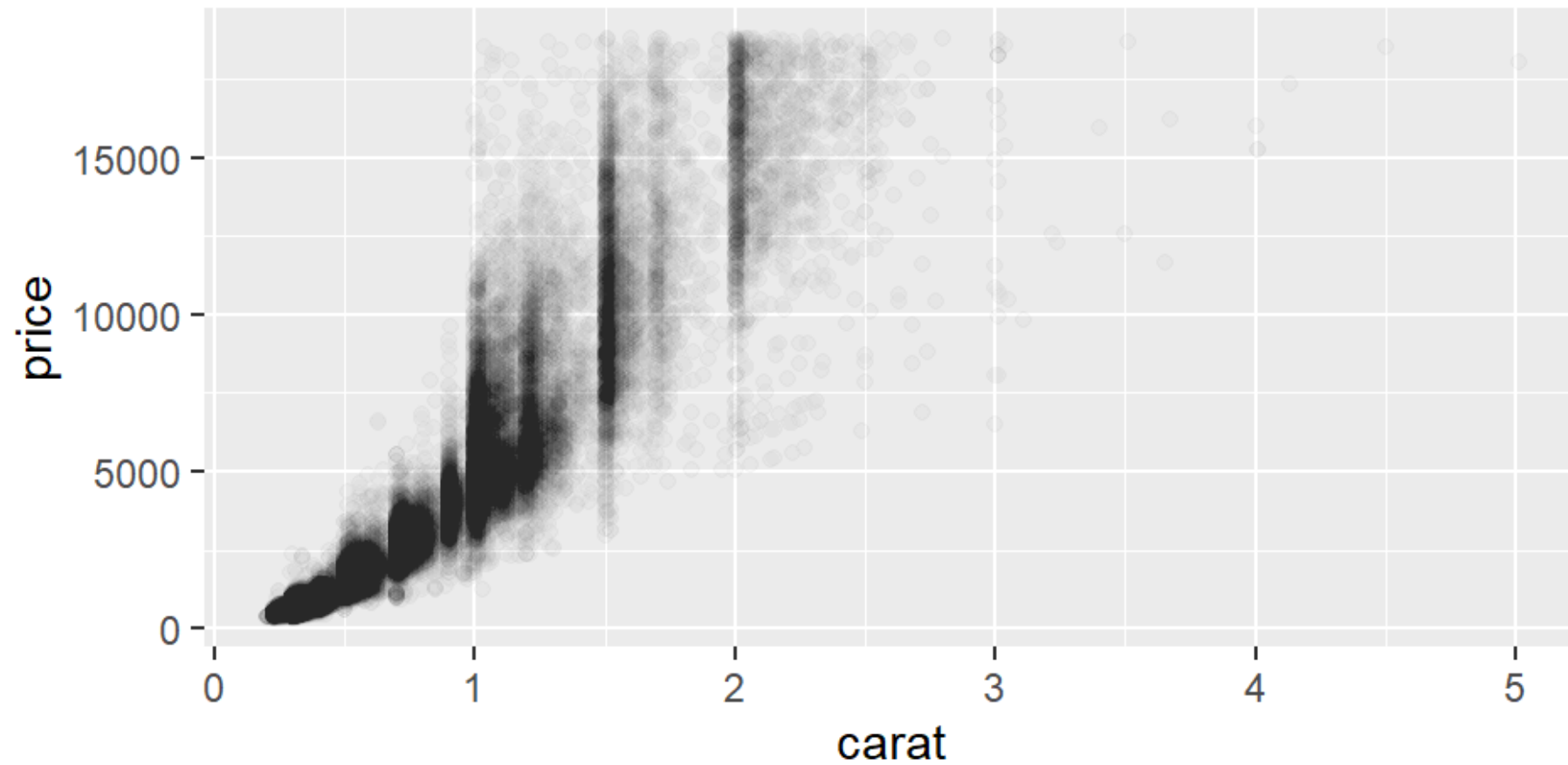
Other Ways to Visualise the Relationship

- Use the `alpha` aesthetic to add transparency
- Use `geom_bin2d()` or `geom_hex()` to bin observations in two dimensions
- Bin one continuous variable so that it behaves like a categorical variable



Example: Add Transparency

```
1 ggplot(data = diamonds) +  
2   geom_point(mapping = aes(x = carat, y = price), alpha = 1 / 100)
```



From Data Patterns to Models

- Patterns in your data provide clues about relationships
- Models are tools for extracting and formalising these patterns



References

Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. 2023. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. 2nd ed. O'Reilly Media.

